Universidad Nacional de La Plata

Facultad de Informática

TRABAJO FINAL

Problema de Difusión de Calor

Taller de Programación sobre GPUs
Prof. Adrián Pousa

Federico Dobal

Índice

Introducción 3

Programa principal (host) 4
- Validación de parámetros . . . 4
- Preparación de la grilla . . . 4
  - Distancia entre las fuentes de calor . . . 4
  - Gestión inicial de memoria . . . 4
  - Fuentes de calor iniciales . . . 5
  - Dimensionamiento de hilos y bloques . . . 5
- Bucle de pasos . . . 6

Kernel de difusión (device) 8
- Declaración del tile . . . 8
- Identificación del hilo . . . 8
- Implementación del halo . . . 9
- Lectura coalescente . . . 9
- Sincronización de hilos . . . 10
- Actualización de las temperaturas . . . 10

Resultados 11
- Medición de tiempos . . . 11
- Tiempos obtenidos . . . 11
- Análisis de rendimiento . . . 13

Versión con CUDA Streams 15
- Motivación . . . 15
- Implementación . . . 15
- Resultados . . . 16

Uso de IA 18

Visualización de la simulación 21

Conclusión 24

Introducción

La conducción de calor sucede desde un cuerpo con mayor temperatura hacia otro con menor temperatura. Este fenómeno está dado por la Ley de Fourier, donde la tasa de cambio de temperatura en un punto depende de la diferencia con sus vecinos.

$$\frac{\partial T}{\partial t} = D\,\nabla^2 T \tag{1}$$

Donde:
T : temperatura
D : coeficiente de difusión térmica
t : tiempo
$\nabla^2 T$: laplaciano de la temperatura (suma de segundas derivadas espaciales)

Objetivo

Simular en GPU la difusión de calor sobre una superficie (Ley de Fourier). La superficie se representa con una grilla de N $\times$ N celdas, donde cada celda guarda la temperatura $T(x, y)$ de un punto, y en cada paso se actualiza según:

$$\begin{gathered} T^{(t+1)}(x,y) = T^{(t)}(x,y) \\ +D\big[T(x+1,y) + T(x-1,y) + T(x,y+1) + T(x,y-1) - 4\,T(x,y)\big] \end{gathered} \tag{2}$$

Programa principal (host)

```
9  #define N_REFERENCIA 512
10 #define DISTANCIA_REFERENCIA 20
11 #define TEMP_FUENTE 100.0f
12 #define DIFUSION 0.25f
13 #define ANCHO_BLOQUE 32
```

Macros del programa

Validación de parámetros

```
15 bool validar_parametros(int argc, char *argv[])
```

Esta función se asegura de que:

- se pasen los tres parámetros <N><Pasos de simulación><hilos por bloque>;
- N sea potencia de 2 [1];
- N sea mayor o igual a 32 [2];
- se ingrese una cantidad de pasos mayor o igual a 1;
- los hilos por bloque sean múltiplos de 32, entre 32 y 1024 [3].

Preparación de la grilla

Distancia entre las fuentes de calor

En el algoritmo secuencial, existe una fuente de calor en el centro de la grilla y 4 más en diagonal, a una distancia fija de 20 celdas en x y en y.
En esta implementación paralela decidí que esa distancia no sea fija ya que, en una grilla grande, las fuentes quedan muy juntas y, en una grilla muy chica, no entran. De esta forma, se sigue respetando la proporción de 512 celdas $\longleftrightarrow$ distancia 20 del programa secuencial:

```
161 int distancia = N * DISTANCIA_REFERENCIA / N_REFERENCIA;
```

Gestión inicial de memoria

1. Calcular la cantidad total de bytes que ocupará la grilla:

```
163 size_t bytes = (size_t)N * N * sizeof(float);
```

2. Reservar memoria en el host para la grilla:

[1] Requerimiento del enunciado.
[2] Para que las fuentes de calor respeten la proporcionalidad del algoritmo secuencial ya dado sin estar tan juntas entre sí.
[3] Las GPUs NVIDIA, desde compute capability 2.0, aceptan un máximo de 1024 hilos por bloque.

```
164 float *h_grilla = (float *)malloc(bytes);
```

Una grilla de dos dimensiones se guarda en un arreglo de una dimensión, fila tras fila (ordenamiento por filas).

3. Reservar memoria en GPU para las grillas:

```
166 float *d_grilla;
167 HANDLE_ERROR(cudaMalloc(&d_grilla, bytes));
168 float *d_nueva;
169 HANDLE_ERROR(cudaMalloc(&d_nueva, bytes));
```

4. Utilizo cudaMemset para evitar la transferencia de la grilla del host hacia las grillas en GPU, haciendo que esta las inicialice directamente:

```
181 HANDLE_ERROR(cudaMemset(d_grilla, 0, bytes));
182 HANDLE_ERROR(cudaMemset(d_nueva, 0, bytes));
```

Fuentes de calor iniciales

La inicialización de las 5 fuentes de calor en 100 la lleva a cabo un kernel (directamente la GPU), y el host solo recibe copias para guardar los frames:

```
49 __global__ void inicializar_fuentes_de_calor(float *grilla, int N, int distancia) {
50     int centroX = N / 2;
51     int centroY = N / 2;
52 
53     grilla[centroY * N + centroX] = TEMP_FUENTE;
54     grilla[(centroY + distancia) * N + (centroX + distancia)] = TEMP_FUENTE;
55     grilla[(centroY - distancia) * N + (centroX + distancia)] = TEMP_FUENTE;
56     grilla[(centroY + distancia) * N + (centroX - distancia)] = TEMP_FUENTE;
57     grilla[(centroY - distancia) * N + (centroX - distancia)] = TEMP_FUENTE;
58 }
```

Dimensionamiento de hilos y bloques

- **Dimensiones de bloque:** este ancho de bloque está fijado en 32 hilos para que coincida con el tamaño del warp. De esta forma, cada fila del bloque corresponde exactamente a un warp, donde sus hilos contiguos ocupan celdas contiguas de dicha fila. Al ser 32 celdas de 4 bytes (float) por fila, cada warp accede a 128 bytes contiguos que coinciden exactamente con el tamaño de segmento de memoria global para palabras de 4 bytes.
  Por otro lado, al ser N potencia de 2 y mayor o igual a 32, los accesos del warp quedan alineados con cada segmento (o sea, que empiezan en una dirección múltiplo de 128). Así, cada lectura o escritura de los bloques se resuelve de forma coalescente.

  Este mismo ancho también sirve para el acceso a memoria compartida, que se divide en 32 bancos de palabras de 4 bytes c/u. Como los 32 hilos de cada warp acceden a 32 floats consecutivos de una misma fila, cada uno accede a un banco distinto y el acceso se resuelve sin necesidad de serializar (lo cual costaría hasta 32 accesos por warp).

	Bloque	
hilosXBloque	Ancho	Alto
32	32	$\frac{32}{32} = 1$
64	32	$\frac{64}{32} = 2$
128	32	$\frac{128}{32} = 4$
256	32	$\frac{256}{32} = 8$
512	32	$\frac{512}{32} = 16$
1024	32	$\frac{1024}{32} = 32$

```
192 dim3 bloque(ANCHO_BLOQUE, hilosXBloque / ANCHO_BLOQUE);
```

- **Dimensiones de grid:** hacen falta suficientes bloques para cubrir las N columnas y filas. En algunos casos las filas pueden sobrar, y esos son los hilos que no van a operar sobre ninguna celda de la grilla:

		Bloque		Grid		
N	hilosXBloque	Ancho	Alto	Ancho	Alto	Hilos totales
32	256	32	$\frac{256}{32} = 8$	$\lceil \frac{32}{32} \rceil = 1$	$\lceil \frac{32}{8} \rceil = 4$	32×32 (justo)
	96	32	$\frac{96}{32} = 3$	$\lceil \frac{32}{32} \rceil = 1$	$\lceil \frac{32}{3} \rceil = 11$	$32 \times \mathbf{33}$ (sobra 1 fila)
64	128	32	$\frac{128}{32} = 4$	$\lceil \frac{64}{32} \rceil = 2$	$\lceil \frac{64}{4} \rceil = 16$	64×64 (justo)
	160	32	$\frac{160}{32} = 5$	$\lceil \frac{64}{32} \rceil = 2$	$\lceil \frac{64}{5} \rceil = 13$	$64 \times \mathbf{65}$ (sobra 1 fila)
128	512	32	$\frac{512}{32} = 16$	$\lceil \frac{128}{32} \rceil = 4$	$\lceil \frac{128}{16} \rceil = 8$	128×128 (justo)
	192	32	$\frac{192}{32} = 6$	$\lceil \frac{128}{32} \rceil = 4$	$\lceil \frac{128}{6} \rceil = 22$	$128 \times \mathbf{132}$ (sobran 4 filas)
256	1024	32	$\frac{1024}{32} = 32$	$\lceil \frac{256}{32} \rceil = 8$	$\lceil \frac{256}{32} \rceil = 8$	256×256 (justo)
	224	32	$\frac{224}{32} = 7$	$\lceil \frac{256}{32} \rceil = 8$	$\lceil \frac{256}{7} \rceil = 37$	$256 \times \mathbf{259}$ (sobran 3 filas)
512	256	32	$\frac{256}{32} = 8$	$\lceil \frac{512}{32} \rceil = 16$	$\lceil \frac{512}{8} \rceil = 64$	512×512 (justo)
	96	32	$\frac{96}{32} = 3$	$\lceil \frac{512}{32} \rceil = 16$	$\lceil \frac{512}{3} \rceil = 171$	$512 \times \mathbf{513}$ (sobra 1 fila)
1024	512	32	$\frac{512}{32} = 16$	$\lceil \frac{1024}{32} \rceil = 32$	$\lceil \frac{1024}{16} \rceil = 64$	1024×1024 (justo)
	640	32	$\frac{640}{32} = 20$	$\lceil \frac{1024}{32} \rceil = 32$	$\lceil \frac{1024}{20} \rceil = 52$	$1024 \times \mathbf{1040}$ (sobran 16 filas)

```
193 dim3 grid((N + bloque.x - 1) / bloque.x, (N + bloque.y - 1) / bloque.y);
```

Bucle de pasos

En cada paso primero se guarda en disco el estado actual de la grilla y después se lanza el kernel, que calcula el paso siguiente en d_nueva. Para que ese resultado sea la entrada del próximo paso no copio

nada, sino que intercambio los punteros, así d_grilla pasa a apuntar a la grilla recién calculada y d_nueva a la vieja, que se va a sobrescribir.

Por último copio la grilla al host para poder guardarla en la siguiente vuelta. En el último paso esa copia no se hace, porque ya no queda ningún frame por guardar. Las líneas que omití en el fragmento son las de medición de tiempos.

```
196 for (int paso = 0; paso < pasos; paso++) {
198     guardar_paso(h_grilla, paso, N);
199     ...
202     difusion<<<grid, bloque, bytes_compartidos>>>(d_grilla, d_nueva, N, distancia);
203     ...
209     float *aux = d_grilla;
210     d_grilla = d_nueva;
211     d_nueva = aux;
212
213     if (paso + 1 < pasos) {
214         ...
215         HANDLE_ERROR(cudaMemcpy(h_grilla, d_grilla, bytes, cudaMemcpyDeviceToHost));
216         ...
217     }
218 }
```

Kernel de difusión (device)

Este es el kernel lanzado en cada paso del bucle principal. Se puede descomponer en seis pasos críticos.

Declaración del tile

Antes de optimizar el algoritmo con el uso de memoria compartida, cada hilo debía leer 5 veces de memoria global por cada paso de la simulación:

- una vez para sí mismo;
- cuatro veces, una por cada vecino (arriba, abajo, izquierda, derecha).

La memoria global es la más lenta de la jerarquía de memoria de la GPU. Para poder aprovechar entonces la memoria compartida, que es mucho más rápida por estar ubicada dentro del Streaming Multiprocessor, hago uso del tile.

```
106 extern __shared__ float tile[];
```

Definición de tile

El tile es la porción de grilla que cada bloque trae a su memoria compartida. Este es visible para los hilos de ese bloque.

- Extern: indica que se declara sin tamaño, y el tamaño en bytes se pasa al kernel como tercer parámetro de la configuración de ejecución.
- __shared__: la variable vive en memoria compartida.

```
194 size_t bytes_compartidos = (bloque.x + 2) * (bloque.y + 2) * sizeof(float);
195 ...
196 for (int paso = 0; paso < pasos; paso++) {
197     ...
202     difusion<<<grid, bloque, bytes_compartidos>>>(d_grilla, d_nueva, N, distancia);
203     ...
217 }
```

Más adelante está la explicación sobre por qué se le suma 2 al ancho y al alto.

Identificación del hilo

Obtener el idx del hilo en la grilla.

- ancho y alto del bloque: blockDim.x y blockDim.y;
- columna y fila del hilo dentro del bloque: threadIdx.x y threadIdx.y;
- columna y fila del hilo en toda la grilla:
  - columna: blockIdx.x $\times$ ancho_bloque + col_bloque;

- fila: blockIdx.y $\times$ alto_bloque + fila_bloque;

- identificador final del hilo: $fila * N + col$.

Implementación del halo

Los hilos del borde del bloque tienen vecinos que pertenecen a otros bloques y, por lo tanto, no están en el tile. Para que todos los hilos encuentren sus cuatro vecinos en memoria compartida, se agrega al tile una fila extra arriba y abajo, y una columna extra a cada lado, por lo que pasa a medir $(ancho+2)\times(alto+2)$. A estas celdas extra se las llama halo. No tienen hilos propios, sino que las cargan los hilos del borde del bloque. Así, el cálculo de todos los hilos lee únicamente de memoria compartida, sin casos especiales para el borde. Por esta razón, en el programa principal se le suma 2 al ancho y al alto al calcular la cantidad de bytes compartidos.

```
117 int ancho_tile = ancho_bloque + 2;
118 int col_tile = col_bloque + 1;
119 int fila_tile = fila_bloque + 1;
```

Lectura coalescente

Habiendo organizado los hilos de tal forma que cada fila de bloque corresponde a un warp completo (32 hilos), y que el índice idx avanza consecutivamente por columnas, la lectura de memoria global de las celdas del bloque será coalescente.

Cada hilo guarda en su posición del tile el valor de su posición en la grilla. Por cada fila de bloque (32 hilos consecutivos) habrá una sola transacción desde memoria global.

```
123 if (dentro_grilla) {   ← col <N && fila <N → para descartar los hilos que sobran
124     tile[fila_tile * ancho_tile + col_tile] = d_grilla[idx];
125     ...
```

Luego, los hilos del borde del bloque cargan además las celdas del halo. Cada warp, que corresponde a una fila del bloque, carga sus propias celdas en una transacción, y las celdas laterales del halo en una transacción cada una, leídas por separado por los hilos 0 y 31. Así, los warps intermedios requieren tres transacciones.

Además, el primer y el último warp cargan la fila de halo superior o inferior, que los 32 hilos leen de forma coalescente en una única transacción, por lo que estos dos warps requieren cuatro transacciones cada uno. Los vecinos de arriba y de abajo de los warps intermedios no se cargan por separado, ya que son celdas de los warps vecinos del mismo bloque.

```
125     ...
126     if (fila_bloque == 0 && fila > 0)
127       tile[0 * ancho_tile + col_tile] = d_grilla[(fila - 1) * N + col];
128     if (fila_bloque == alto_bloque - 1 && fila < N - 1)
129       tile[(alto_bloque + 1) * ancho_tile + col_tile] = d_grilla[(fila + 1) * N + col];
130     if (col_bloque == 0 && col > 0)
131       tile[fila_tile * ancho_tile + 0] = d_grilla[fila * N + (col - 1)];
132     if (col_bloque == ancho_bloque - 1 && col < N - 1)
133       tile[fila_tile * ancho_tile + (ancho_bloque + 1)] = d_grilla[fila * N + (col + 1)];
134 }
```

Sincronización de hilos

Es imprescindible que ningún hilo avance al cálculo hasta que todos los hilos del bloque hayan terminado de cargar el tile. Cada hilo lee del tile los valores de sus vecinos, que fueron cargados por otros hilos, posiblemente de otros warps cuyo orden de ejecución se desconoce. Sin una barrera intermedia, un hilo podría leer una celda del tile antes de que fuera cargada. Además, esta directiva se pone fuera del if(dentro_grilla) para que los hilos que sobran también puedan llegar a este punto y no bloquear a los hilos que sí sirven.

```
...
__syncthreads();
...
```

Actualización de las temperaturas

Ahora, con la misma fórmula fila_tile $\times$ ancho_tile $+$ col_tile, se procede a leer de memoria compartida el valor de su celda y de sus vecinos para sumarlos, aplicar la fórmula de difusión para el paso siguiente, y guardarlo en la grilla nueva.

Nótese que solo se actualizarán aquellas celdas que no corresponden a fuentes de calor y, en caso contrario, la fuente se mantendrá en 100 grados. Para eso uso el valor entero (0 o 1) que devuelve la función es_fuente. Luego se calculan los dos valores posibles y cada uno se multiplica por ese entero o por su negación.

$$\text{d_nueva[idx]} = \text{cálculo} \times \neg\,\text{es_fuente} + 100 \times \text{es_fuente}$$

De esta manera, todos los hilos ejecutarán lo mismo y se evitará la divergencia que supondría una ejecución condicional. Tampoco se actualizará el borde de la grilla, el cual se mantiene a 0 grados.

```
if (!frontera) {   ← col == 0 || col >= N - 1 || fila == 0 || fila >= N - 1 → borde de la grilla
    int int_es_fuente = es_fuente(col, fila, N, distancia);

    float centro = tile[fila_tile * ancho_tile + col_tile];
    float arriba = tile[(fila_tile - 1) * ancho_tile + col_tile];
    float abajo = tile[(fila_tile + 1) * ancho_tile + col_tile];
    float izq = tile[fila_tile * ancho_tile + (col_tile - 1)];
    float der = tile[fila_tile * ancho_tile + (col_tile + 1)];

    float suma_vecinos = arriba + abajo + izq + der;

    d_nueva[idx] = (centro + DIFUSION * (suma_vecinos - 4 * centro)) * !int_es_fuente +
        TEMP_FUENTE * int_es_fuente;
}
```

Resultados

Medición de tiempos

Utilicé Claude Opus 5.5 para crear dos scripts que realizan un barrido de parámetros en el cluster, uno para la versión secuencial y otro para la versión CUDA. Cada configuración se ejecutó 3 veces con 100 pasos en el nodo hexa1, y se informa el promedio. Los frames se escribieron en el disco local del nodo.

Las dos versiones se miden de la misma forma, con dwalltime dentro del programa, desde la inicialización de la grilla hasta el final del bucle. Quedan afuera la creación del directorio de salida, las impresiones por pantalla y, en la versión CUDA, la reserva de memoria, donde se inicializa CUDA. Al código secuencial solo le agregué la medición.

Además del tiempo total, en el secuencial mido el cálculo y la escritura de los frames, y en la versión CUDA los kernels (con eventos), las copias D2H y la escritura. El speedup lo calculo sobre el tiempo total, porque la escritura de los frames es parte del problema y la hacen las dos versiones. Con el criterio de la teoría, que toma solo el kernel y las transferencias (tabla 4), el speedup sería 0.92, 0.96, 1.34 y 2.51.

Tiempos obtenidos

Versión secuencial

N	Rep. 1 (s)	Rep. 2 (s)	Rep. 3 (s)	Promedio (s)	Cálculo (ms/paso)	Disco (s)
512	0.0201	0.0209	0.0208	0.0206	0.0784	0.0126
1024	0.0746	0.0652	0.0628	0.0675	0.2470	0.0424
2048	0.3186	0.3194	0.3163	0.3181	1.2499	0.1908
4096	2.0424	1.7743	1.7813	1.8660	9.1706	0.9404

Tabla 1: Tiempos de la versión secuencial (100 pasos). El cálculo y el disco son el promedio de las 3 repeticiones.

Versión CUDA

N	Hilos/bloque	Total (s)	Kernel (ms/paso)	D2H (s)	Disco (s)	Ancho de banda (GB/s)
512	64	0.0272	0.0077	0.0082	0.0173	273.1
	128	0.0260	0.0072	0.0081	0.0161	293.1
	256	0.0261	0.0068	0.0079	0.0163	310.1
	512	0.0257	0.0083	0.0080	0.0162	253.4
	1024	0.0262	0.0101	0.0080	0.0163	208.9
1024	64	0.0905	0.0202	0.0236	0.0630	415.7
	128	0.0918	0.0189	0.0237	0.0644	445.1
	256	0.0872	0.0198	0.0234	0.0601	423.9
	512	0.0918	0.0208	0.0236	0.0644	403.2
	1024	0.0926	0.0293	0.0237	0.0642	286.8
2048	64	0.3289	0.0944	0.0842	0.2312	355.5
	128	0.3389	0.0938	0.0841	0.2412	357.7
	256	0.3404	0.0973	0.0843	0.2424	345.1
	512	0.3419	0.1021	0.0841	0.2435	328.8
	1024	0.3590	0.1644	0.0840	0.2543	204.0
4096	64	1.2982	0.3672	0.3282	0.9200	365.5
	128	1.3377	0.3721	0.3291	0.9582	360.8
	256	1.3930	0.3846	0.3292	1.0123	349.0
	512	1.3025	0.4043	0.3295	0.9191	332.0
	1024	1.3638	0.6462	0.3286	0.9572	207.7

Tabla 2: Tiempos de la versión CUDA (100 pasos, promedio de 3 repeticiones).

Speedup

Para cada N tomo la configuración de hilos por bloque con menor tiempo de kernel.

N	Hilos/bloque	Secuencial (s)	CUDA (s)	Speedup
512	256	0.0206	0.0261	0.79
1024	128	0.0675	0.0918	0.74
2048	128	0.3181	0.3389	0.94
4096	64	1.8660	1.2982	1.44

Tabla 3: Speedup del tiempo total (100 pasos, promedio de 3 repeticiones).

Análisis de rendimiento

Dónde se va el tiempo

Ninguna de las dos versiones está dominada por el cálculo.[4] En el secuencial la escritura de los frames ocupa entre el 50 y el 63 % del tiempo. En la versión CUDA los kernels ocupan menos del 3 %, las copias D2H alrededor del 25 al 30 % y la escritura entre el 62 y el 71 %, y todo ocurre en serie. Por eso la configuración del kernel casi no influye en el total. Con N = 4096, pasar de la peor configuración de hilos a la mejor ahorra unos 28 ms de kernel, apenas el 2 % del tiempo total.

N	Cálculo sec. (s)	Kernels + D2H (s)	Escritura sec. (s)	Escritura CUDA (s)
512	0.0079	0.0086	0.0126	0.0163
1024	0.0247	0.0256	0.0424	0.0644
2048	0.1250	0.0935	0.1908	0.2412
4096	0.9171	0.3650	0.9404	0.9200

Tabla 4: Tiempo de cálculo y de escritura de cada versión (100 pasos, promedio de 3 repeticiones).

Comportamiento del speedup

En el cálculo, hasta N = 1024 la copia de la grilla desde la GPU ya cuesta lo mismo que calcularla en la CPU. Recién desde N = 2048 la GPU ahorra tiempo, y con N = 4096 ahorra 0.55 s, porque el costo por celda del secuencial pasa de unos 0.3 ns a 0.55 ns, probablemente porque las grillas (128 MB) dejan de entrar en la caché. A esto se suma que el secuencial copia new_grid a grid en cada paso, mientras que la versión CUDA solo intercambia punteros.

En la escritura, la versión CUDA es entre un 26 y un 52 % más lenta hasta N = 2048, posiblemente porque en el secuencial la grilla recién calculada todavía está en caché. Con N = 4096 las dos tardan lo mismo.

Hasta N = 2048 lo que se pierde en la escritura supera lo que se gana en el cálculo. Con N = 4096 la escritura se iguala y el speedup llega a 1.44.

Efecto de los hilos por bloque

Entre 64 y 256 hilos por bloque el tiempo del kernel varía menos del 5 % desde N = 2048. Con 1024 hilos es entre 1.49 y 1.76 veces más lento, porque un SM admite como máximo 1536 hilos y entra un solo bloque, con una ocupación del 67 %.

Calculo el ancho de banda efectivo con la fórmula de la teoría, $BW = (RB + WB)/(t \cdot 10^9)$: en cada paso el kernel lee y escribe una grilla ($RB = WB = 4N^2$ bytes) y t es su tiempo. El máximo teórico de la placa es de 448 GB/s (28 Gbps $\times$ 128 bits/8). Con N $\leq$ 1024 el valor medido se acerca a ese máximo, probablemente porque las dos grillas entran en la caché L2. Con grillas más grandes el kernel lee de la memoria de la placa y queda en unos 360 GB/s, cerca del 80 % del máximo, así que está limitado por la memoria.

[4] Para este análisis usé IA para identificar patrones, comportamientos y aspectos ocultos en los resultados, uso permitido por las pautas de uso de IA de la materia.

Aspectos ocultos

- La columna de disco no mide el disco. Se escribe a más de 6 GB/s porque los frames solo se copian a la caché de páginas del sistema operativo.
- La escritura de la versión CUDA tiene dos niveles (con N = 4096, 0.87 s o 1.01 s) que no dependen de los hilos por bloque. Por eso la mejor configuración la elegí según el tiempo de kernel y no según el total.
- Las copias D2H alcanzan solo entre 13 y 20 GB/s, porque la memoria del host es paginable y el driver hace una copia intermedia.

Límite impuesto por la escritura

Aunque el cálculo y las copias de la versión CUDA fueran instantáneos, su tiempo no podría bajar del tiempo de escritura. Por la ley de Amdahl, esto limita el speedup a 1.26, 1.05, 1.32 y 2.03 para cada tamaño.

Versión con CUDA Streams

Motivación

En la versión CUDA las copias D2H ocupan entre el 25 y el 30 % del tiempo y la escritura entre el 62 y el 71 %, y las dos ocurren en serie con el kernel. Surge entonces la pregunta vista en clase: ¿es posible ocultar la latencia de estas operaciones? La idea es que mientras la CPU escribe un frame, la GPU calcule y copie el siguiente por porciones, de modo que la copia de una porción se haga al mismo tiempo que el cálculo de la siguiente. Para eso CUDA provee los streams.

Definición de stream

Un stream es una cola de trabajo para el device, donde el host encola el trabajo y continúa inmediatamente.

Implementación

Sigo el esquema visto en clase (superposición kernel-D2H). Creo un vector de streams donde cada uno trabaja sobre su porción de los datos, de modo que una invocación al kernel se reemplaza por varias y cada una se superpone con su copia D2H. Divido la grilla en STREAMS = 5 bandas de filas. Lo combino con la alternativa híbrida que superpone el trabajo de la GPU con el de la CPU, que acá lo tomo como la escritura a disco.

Reservo dos grillas en el host con cudaMallocHost, porque cudaMemcpyAsync necesita memoria pinned, y así la GPU copia un frame en una mientras la CPU escribe la otra a disco.

El kernel recibe un puntero al comienzo de su banda y las filas donde empieza y termina. La fila global la sigue necesitando para detectar la frontera y las fuentes, y solo escribe las filas de su banda. El halo se lee igual que antes, porque las filas vecinas están en la misma reserva, antes o después del puntero.

```
111 __global__ void difusion(float *d_grilla, float *d_nueva, int N, int distancia, int
        fila_inicio, int fila_fin) {
112     ...
121     int fila_banda = blockIdx.y * alto_bloque + fila_bloque;
122     int fila = fila_inicio + fila_banda;
123     int idx = fila_banda * N + col;
124     ...
146     if (!frontera && fila < fila_fin) {
```

En cada paso, cada stream encola el kernel y la copia de su banda del frame siguiente, y mientras la GPU trabaja el host escribe a disco el frame actual. Recién al final espera a la GPU.

```
214 for (int paso = 0; paso < pasos; paso++) {
215     ...
219     if (hay_siguiente) {
220         for (int i = 0; i < STREAMS; i++) {
221             int fila_inicio = i * N / STREAMS;
222             int fila_fin = (i + 1) * N / STREAMS;
223             size_t offset = (size_t)fila_inicio * N;
```

```
224             ...
228             difusion<<<grid_banda, bloque, bytes_compartidos, streams[i]>>>(&d_grilla[
                    actual][offset], &d_grilla[siguiente][offset], N, distancia, fila_inicio,
                    fila_fin);
232             ...
233             HANDLE_ERROR(cudaMemcpyAsync(&h_grilla[siguiente][offset], &d_grilla[siguiente
                    ][offset], bytes_banda, cudaMemcpyDeviceToHost, streams[i]));
235             ...
236         }
237     }
238     ...
240     guardar_paso(h_grilla[actual], paso, N);
241     ...
244     HANDLE_ERROR(cudaDeviceSynchronize());
245     ...
255 }
```

Ningún buffer se escribe mientras otra operación lo usa: en un paso todas las bandas leen la misma grilla y escriben filas distintas de la otra, y la copia va a la grilla del host que la CPU no está escribiendo. Como por el halo cada banda lee filas que calculan otros streams, el cudaDeviceSynchronize del final asegura que el paso anterior terminó completo antes de lanzar el siguiente.

Resultados

Verifiqué que los frames sean idénticos byte a byte a los de la versión sin streams, para N = 512, 1024 y 2048.

```
== Version sin streams (referencia): N=1024 pasos=30 hilos=256
== Version con streams: difusion_calor_cuda_streams.cu
...
Frames identicos (30 archivos)
```

Volví a medir el secuencial y la versión sin streams en la misma sesión, con 100 pasos, 3 repeticiones y los mismos hilos por bloque, por eso los tiempos no coinciden exactamente con los de antes. Además mido cuánto espera el host a la GPU.

N	Hilos/bloque	Sin streams (s)	Con streams (s)	Mejora	Escritura (s)	Espera (s)
512	256	0.0285	0.0209	1.36	0.0180	0.0001
1024	128	0.0967	0.0719	1.35	0.0688	0.0001
2048	128	0.3629	0.2553	1.42	0.2517	0.0001
4096	64	1.4108	0.9722	1.45	0.9657	0.0002

Tabla 5: Tiempo total de las dos versiones CUDA (100 pasos, promedio de 3 repeticiones). La escritura y la espera son de la versión con streams.

El tiempo total baja entre un 26 y un 31 % y la espera es prácticamente nula, o sea que la GPU queda oculta detrás de la escritura. Desde N = 1024 el tiempo total queda a menos de un 5 % del de escritura. Con N = 512 queda un 16 % por encima, porque manejar 5 streams por paso (encolar kernels y copias y leer sus eventos) tiene un costo fijo en el host que con grillas chicas ya no es despreciable.

N	Secuencial (s)	Speedup sin streams	Speedup con streams	Cota
512	0.0222	0.78	1.06	1.23
1024	0.0719	0.74	1.00	1.05
2048	0.3271	0.90	1.28	1.30
4096	1.9719	1.40	2.03	2.04

Tabla 6: Speedup del tiempo total frente al secuencial. La cota es el tiempo del secuencial dividido el tiempo de escritura con streams.

Desde N = 2048 el speedup queda prácticamente en la cota que impone la escritura, y con N = 512 se aleja por el costo de manejar los streams. Con N = 1024 empata con el secuencial porque la versión CUDA escribe los frames más lento, y la escritura sola ya tarda casi lo mismo que todo el secuencial. Por lo mismo, optimizar la GPU ya no mejora el tiempo total: con N = 4096 da entre 0.972 y 0.978 s para todas las configuraciones de hilos, incluso con 1024, cuyo kernel es 1.76 veces más lento.

La mejora frente a la versión sin streams viene de superponer la escritura a disco con el trabajo de la GPU. Una versión anterior con solo dos streams (uno de cálculo y otro de copia) ya lograba lo mismo. Dividir la grilla en bandas no agrega ganancia en este problema, como se analiza en la siguiente sección de uso de IA.

Uso de IA

Considero que uno de los mayores provechos que le pude sacar a la IA fue para mejorar un aspecto de los streams. Al principio, había pensado los streams como solamente dos, uno para cálculo y otro para copias. Revisando la clase teórica de CUDA Streams, me di cuenta que se podría lograr una mayor optimización haciendo un vector de streams homogéneos, es decir, que hagan las mismas operaciones y no se diferencien entre, por ejemplo, cálculos y copias.

```
182 cudaStream_t stream_calculo, stream_copia;
183 HANDLE_ERROR(cudaStreamCreate(&stream_calculo));
184 HANDLE_ERROR(cudaStreamCreate(&stream_copia));
185 ...
229 HANDLE_ERROR(cudaStreamWaitEvent(stream_copia, ev_fin_kernel[actual], 0));
230 HANDLE_ERROR(cudaEventRecord(ev_inicio_copia, stream_copia));
231 HANDLE_ERROR(cudaMemcpyAsync(h_grilla[siguiente], d_grilla[siguiente],
232                              bytes, cudaMemcpyDeviceToHost,
233                              stream_copia));
234 HANDLE_ERROR(cudaEventRecord(ev_fin_copia, stream_copia));
235
236 HANDLE_ERROR(cudaEventRecord(ev_inicio_kernel[siguiente], stream_calculo));
237 difusion<<<grid, bloque, bytes_compartidos, stream_calculo>>>(
238   d_grilla[siguiente], d_grilla[actual], N, distancia);
239 HANDLE_ERROR(cudaPeekAtLastError());
240 HANDLE_ERROR(cudaEventRecord(ev_fin_kernel[siguiente], stream_calculo));
```

Para mejorarlo y hacerlo como la cátedra, le pedí a Claude Opus 5.5 y a Claude Sonnet 5.5 lo siguiente:

Prompt

Te adjunto un programa CUDA que simula la difusión de calor en una grilla N×N y guarda cada paso de la simulación en un archivo binario. Usa dos streams, uno para el kernel y otro para la copia D2H, coordinados con cudaStreamWaitEvent, con memoria pinned y doble buffer, de modo que la escritura a disco de un paso se superpone con el cálculo y la copia del siguiente. Modificalo para que siga el esquema de la filmina que adjunto, que sería un vector de N (usá 5 para probar) streams, donde cada stream procesa su porción de la grilla, de modo que la copia de una porción se superponga con el cálculo de otra. La escritura a disco se tiene que seguir superponiendo con el trabajo de la GPU. Los archivos generados tienen que quedar idénticos byte a byte a los del programa original. Dejá los mismos tres parámetros y que imprima los tiempos con el mismo formato. Explicá las decisiones que tomaste.

Criterio	Dos streams (mi versión)	Opus	Sonnet
Pensó durante	–	8' 30"	11'
Compila	Si	Si	Si
Frames idénticos a la versión base (N = 1024)	–	Si	Si
Streams	2 (cálculo y copia)	5	5
División de la grilla	No divide	Bandas de filas de bloques completas	Bandas de filas
Cambio en el kernel	–	Recibe la primera fila de bloques de la banda	Recibe la fila inicial y final, y cambia la carga del halo inferior
Sincronización	La copia espera al kernel con cudaStreamWaitEvent	Cadena de eventos entre todos los kernels	Eventos con las dos bandas vecinas
Trabajo de la GPU mientras se escribe un paso	Copia del siguiente y cálculo del subsiguiente	Copia del siguiente y cálculo del subsiguiente	Copia del siguiente y cálculo del subsiguiente
Líneas de código (sin comentarios)	234	291	279
Prompts hasta que funcionó	–	1	1

Tabla 7: Comparación de la versión con dos streams y las soluciones con un vector de streams.

Las dos soluciones generan frames idénticos a los de la versión base y detectaron por su cuenta la dependencia del halo entre bandas, pero la resuelven distinto. Opus alinea las bandas a filas de bloques completas, así el kernel solo necesita saber dónde empieza su banda, y encadena todos los kernels con eventos, por lo que nunca se ejecutan dos a la vez. Sonnet sincroniza cada banda solo con sus dos vecinas, pero modificó más el kernel para cargar el halo inferior, y uno de sus comentarios justifica una dependencia con un argumento incorrecto, aunque sin afectar el resultado.

Opus además agrupa los datos de las bandas en una estructura con funciones propias, lo que acorta el bucle principal a cambio de más código.

La versión que entrego es más liviana que las dos y se acerca más a la de Sonnet. No usa estructuras ni funciones auxiliares y se le pasa al kernel la fila de inicio y de fin de cada banda. A diferencia de ambas, sincroniza con un único cudaDeviceSynchronize por paso en lugar de eventos entre streams, que resuelve el halo de forma más simple y no cuesta tiempo, porque la GPU termina antes que la escritura.

Versión	Total (s)	Escritura (s)	Espera (s)	Kernel (ms/paso)	D2H (s)
Dos streams	1.5465	1.5455	0.0001	0.0317	0.0043
Opus	1.5647	1.5626	0.0001	0.0606	0.0052
Sonnet	1.5854	1.5811	0.0001	0.0639	0.0053

Tabla 8: Tiempos de las tres versiones en una misma ejecución del script de comparación (N = 1024, 30 pasos, 256 hilos por bloque). Opus y Sonnet con 5 streams.

Fuera de la escritura, la versión con dos streams suma 1.0 ms, contra 2.1 ms de Opus y 4.3 ms de Sonnet, lo que es algo esperable ya que en cada paso se encolan cinco kernels y cinco copias en lugar de uno y

una. Al ser una sola ejecución, estas diferencias de pocos milisegundos pueden variar entre corridas. Con el vector de streams el kernel tarda alrededor del doble por paso y la copia un 20 % más, posiblemente por lanzar cinco kernels y cinco copias más chicas en cada paso, pero como el kernel ocupa menos del 0.1 % del tiempo total no se nota.

Asistente	Prompt	Exitosa	Razones y solución
Uso para los requerimientos del enunciado:			
Claude	Generar scripts para un barrido de parámetros en el cluster hexa.	Si	
Claude	Calcular el speedup entre el secuencial y la mejor configuración CUDA.	Parcial	Usó el tiempo total sin aclarar que la teoría toma solo el kernel y las transferencias. Mantuve el tiempo total como criterio principal, justificando por qué, e informé también el speedup con el criterio de la teoría.
Claude	Analizar los resultados e identificar aspectos ocultos.	Parcial	Detectó que el secuencial y la versión CUDA se medían distinto. Agregué la medición dentro del secuencial y repetí los barridos.
Claude	Probar en el cluster cada paso de la versión con streams.	Si	Encontró un error de compilación por el nombre de un evento y que el tiempo del kernel se sumaba dos veces. Corregí los dos.
Claude	Script que compare los frames de la versión con streams contra la versión sin streams.	Si	
Uso para la comparación de soluciones:			
Claude Opus 5.5	Llevar la versión de dos streams a un vector de streams según la teoría (conversación nueva).	Si	Frames idénticos.
Claude Sonnet 5.5	Llevar la versión de dos streams a un vector de streams según la teoría (conversación nueva).	Si	Frames idénticos.

Tabla 9: Interacción con los motores de IA.

Reconozco, de todas formas, que en este problema el vector de streams no aporta una mejora sobre la versión con dos streams. La ganancia frente a la versión sin streams viene de superponer la escritura a disco con el trabajo de la GPU, algo que los dos streams ya lograban. Dividir la grilla en bandas no agrega más: no hay copias H2D en cada paso, la GPU ya quedaba oculta detrás de la escritura, y lo máximo que podría ahorrar superponer la copia de una banda con el cálculo de otra es el tiempo del kernel, unos 0.03 ms por paso. A cambio, cada paso encola cinco kernels y cinco copias en lugar de uno y una, un costo del mismo orden que esa ganancia. El esquema de la teoría conviene cuando las transferencias y el cálculo pesan parecido y están en el camino crítico, que no es el caso cuando el límite es la escritura en la CPU.

Visualización de la simulación

Para verificar visualmente el resultado usé el script provisto por la cátedra (show_blue2red.py), que lee los frames binarios que el programa guarda en cada paso y los anima como un mapa de calor, de azul (0°C) a rojo (100°C). Las capturas corresponden a una simulación lanzada con los siguientes parámetros: N = 256, 200 pasos y 256 hilos por bloque.

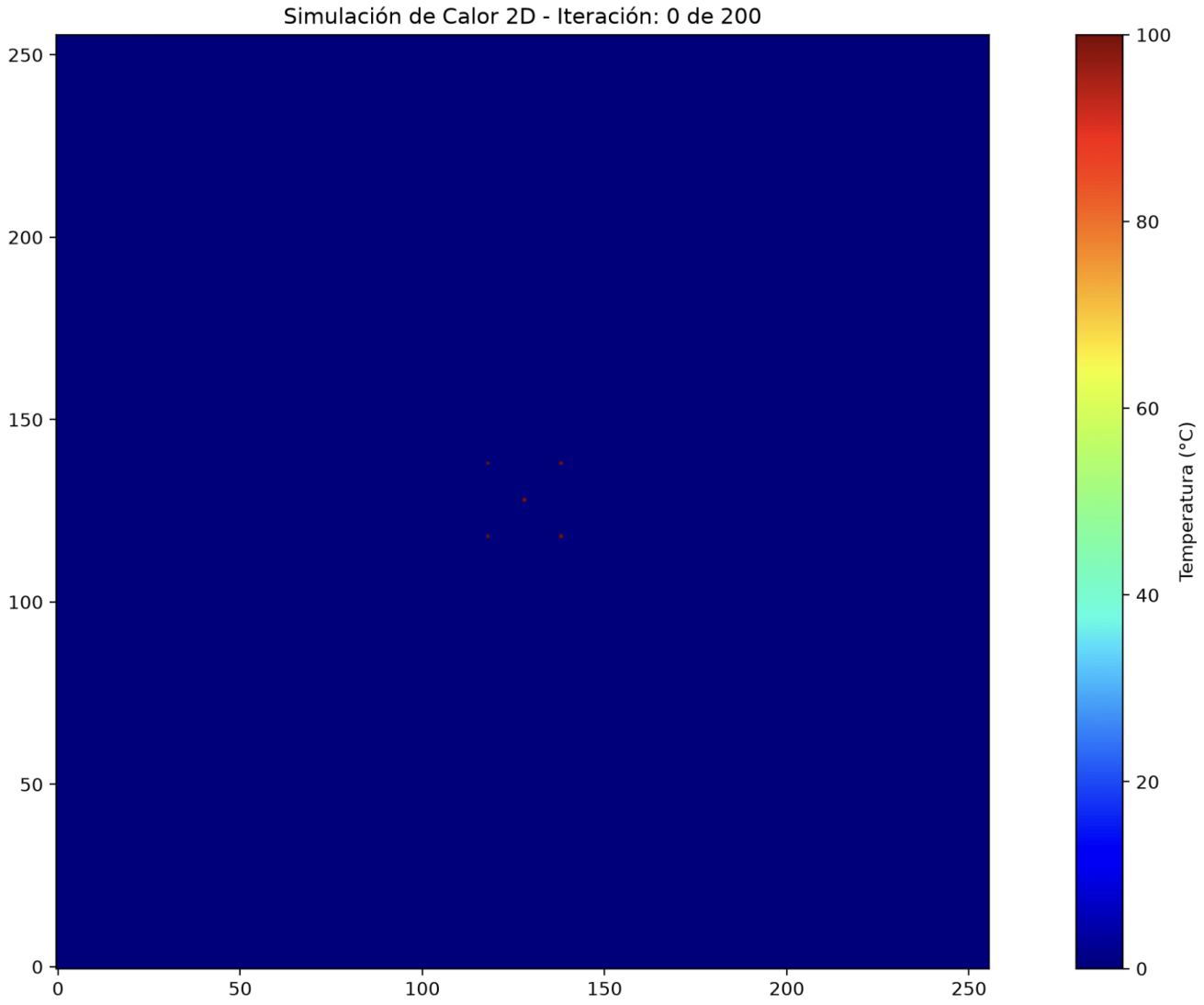


Figura 1: Estado inicial con las 5 fuentes de calor centrales en 100 grados

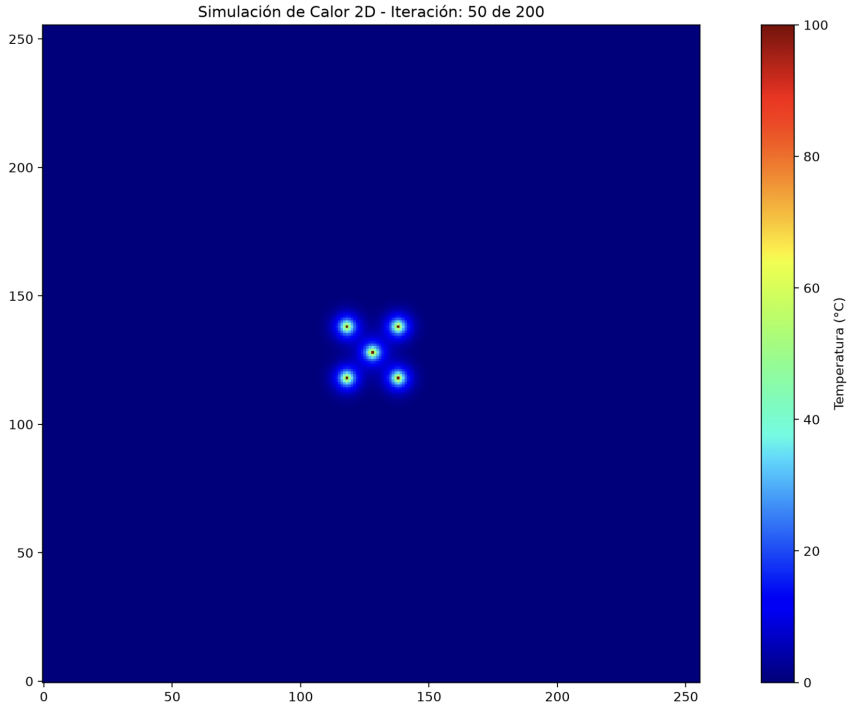


Figura 2: Paso 50

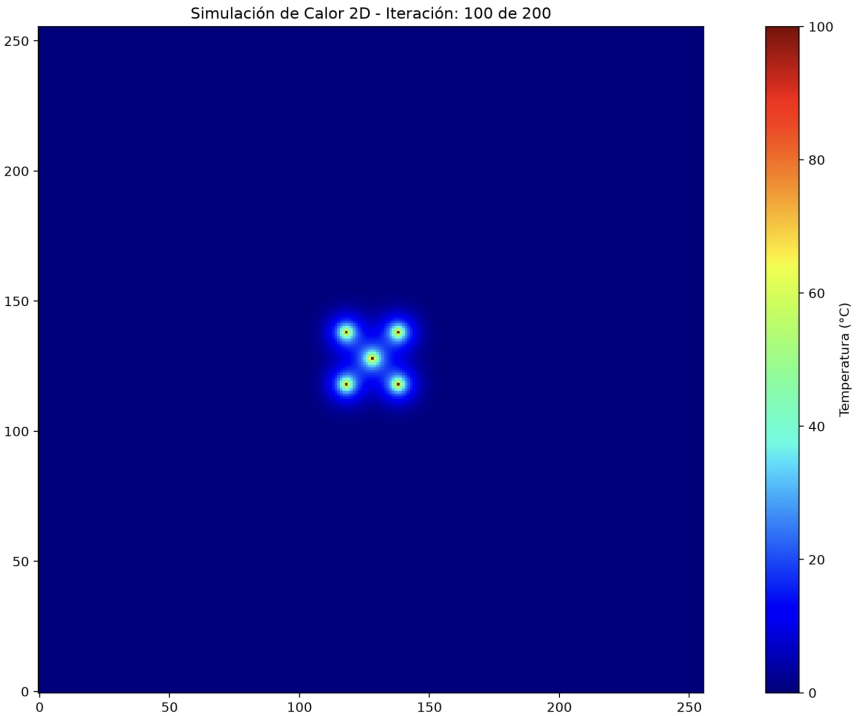


Figura 3: Paso 100

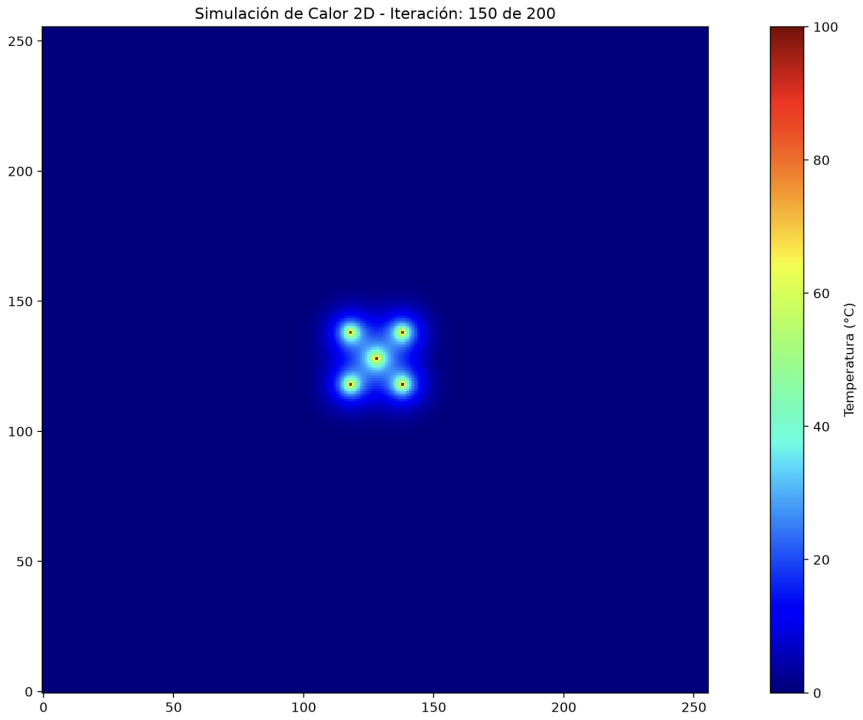


Figura 4: Paso 150

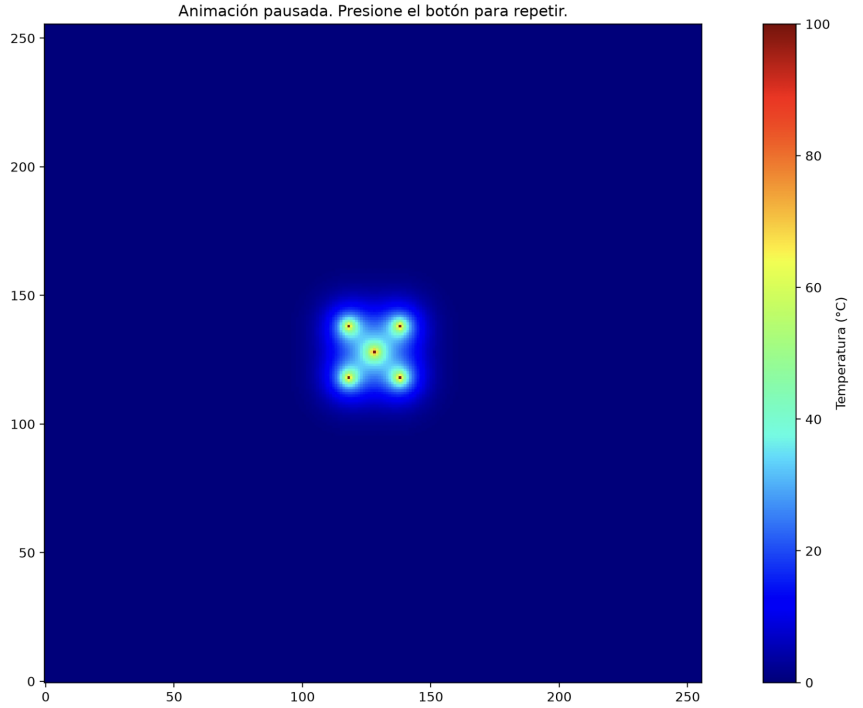


Figura 5: Estado final de la simulación (paso 200)

Conclusión

El presente trabajo me ha sido de gran utilidad para adentrarme en el mundo del cómputo de alto desempeño en GPUs, donde el paralelismo de grano fino es su principal característica.

A partir de la ecuación discretizada de la Ley de Fourier, logré paralelizar la difusión de calor en CUDA, especialmente con el kernel "difusion", donde apliqué la ecuación de lleno junto con lo necesario para aprovechar el tile y la memoria compartida.

Creo que el resultado principal es el cuello de botella que produce la escritura en disco, la cual ocupa arriba del 60 % del tiempo total de ejecución. Esto (ley de Amdahl) explica por qué el speedup termina siendo bajo o, incluso, menor a 1.

Los resultados que obtuve se podrían resumir en principalmente 3:

- El kernel con tiles y lectura coalescente llega a unos 360 GB/s, cerca del 80 % del máximo teórico. O sea, supondría que del lado de la GPU ya no hay mucho más para ganar;
- el tamaño de bloque influye poco, porque el kernel está limitado por la memoria (cerca del 80 % del ancho de banda teórico) y pesa menos del 3 % del tiempo total. Solo 1024 hilos por bloque resulta notablemente más lento;
- con streams y memoria pinned la GPU queda oculta detrás de la escritura, así que el speedup llega prácticamente a la cota desde N = 2048 y la versión CUDA ya no pierde contra el secuencial en ningún tamaño. Para eso alcanzan dos streams, uno de cálculo y otro de copia; el vector de streams de la teoría no agrega mejora.

La principal limitación es la escritura de todos los frames a disco. Como la hace la CPU y no se puede acelerar con la GPU, el tiempo de la versión CUDA no puede bajar del tiempo de escritura, y eso acota el speedup a un máximo cercano a 2 con N = 4096. Pienso que se podría reducir este límite guardando un frame cada k pasos, ya que la escritura se dividiría por k; escribiendo varios frames en paralelo desde distintos hilos de la CPU; o usando un formato más compacto, para tener menos bytes que escribir. También vale la pena probar grillas más grandes, donde el cálculo pesa más frente a la escritura y la GPU tiene más margen para ganar.